

LAB NO#03

Alisha Naushad

(CT-25169)

Question no#01:

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
    Node(int val) {
```

```
        data = val;
```

```
        next = nullptr;
```

```
    }
```

```
};
```

```
class CircularQueue {
```

```
private:
```

```
    Node* rear;
```

```
public:
```

```
    CircularQueue() {
```

```
        rear = nullptr;
```

```
    }
```

```
bool isEmpty() {  
    return rear == nullptr;  
}
```

```
void enqueue(int val) {  
    Node* newNode = new Node(val);  
    if (isEmpty()) {  
        newNode->next = newNode;  
        rear = newNode;  
    } else {  
        newNode->next = rear->next;  
        rear->next = newNode;  
        rear = newNode;  
    }  
    cout << val << " enqueued" << endl;  
}
```

```
void dequeue() {  
    if (isEmpty()) {  
        cout << "Queue Underflow!" << endl;  
        return;  
    }  
}
```

```
Node* front = rear->next;
```

```
if (front == rear) {  
    cout << front->data << " dequeued" << endl;  
    delete front;  
    rear = nullptr;  
} else {  
    cout << front->data << " dequeued" << endl;  
    rear->next = front->next;  
    delete front;  
}  
}  
  
void getFront() {  
    if (isEmpty()) {  
        cout << "Queue is Empty!" << endl;  
    } else {  
        cout << "Front element: " << rear->next->data << endl;  
    }  
}  
  
void display() {  
    if (isEmpty()) {  
        cout << "Queue is Empty!" << endl;  
        return;  
    }  
}
```

```
Node* temp = rear->next;

cout << "Queue elements: ";

do {

    cout << temp->data << " ";

    temp = temp->next;

} while (temp != rear->next);

cout << endl;

}
```

```
~CircularQueue() {

    while (!isEmpty()) {

        dequeue();

    }

}

};
```

```
int main() {

    CircularQueue q;

    q.enqueue(10);

    q.enqueue(20);

    q.enqueue(30);

    q.display();

    q.getFront();

}
```

```
q.dequeue();  
q.display();  
  
q.enqueue(40);  
q.display();  
  
return 0;  
}
```

A screenshot of a terminal window showing the output of a C++ program. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active, displaying the following output:

```
10 enqueued  
20 enqueued  
30 enqueued  
Queue elements: 10 20 30  
Front element: 10  
10 dequeued  
Queue elements: 20 30  
40 enqueued  
Queue elements: 20 30 40  
20 dequeued  
30 dequeued  
40 dequeued  
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 3(CT-25169)>
```

Question no#02:

```
#include <iostream>  
  
using namespace std;  
  
struct Node {  
    int data;  
    Node* prev;  
    Node* next;
```

```
Node(int val) {  
    data = val;  
    prev = nullptr;  
    next = nullptr;  
}  
};
```

```
class Deque {  
private:  
    Node* front;  
    Node* rear;
```

```
public:  
    Deque() {  
        front = nullptr;  
        rear = nullptr;  
    }
```

```
    bool isEmpty() {  
        return front == nullptr;  
    }
```

```
    void insertFront(int val) {  
        Node* newNode = new Node(val);  
        if (isEmpty()) {  
            front = rear = newNode;
```

```
    } else {  
        newNode->next = front;  
        front->prev = newNode;  
        front = newNode;  
    }  
    cout << val << " inserted at front" << endl;  
}
```

```
void insertRear(int val) {  
    Node* newNode = new Node(val);  
    if (isEmpty()) {  
        front = rear = newNode;  
    } else {  
        newNode->prev = rear;  
        rear->next = newNode;  
        rear = newNode;  
    }  
    cout << val << " inserted at rear" << endl;  
}
```

```
void deleteFront() {  
    if (isEmpty()) {  
        cout << "Deque Underflow!" << endl;  
        return;  
    }  
    Node* temp = front;
```

```
cout << temp->data << " deleted from front" << endl;

front = front->next;

if (front == nullptr) {
    rear = nullptr;
} else {
    front->prev = nullptr;
}

delete temp;
}
```

```
void deleteRear() {
    if (isEmpty()) {
        cout << "Deque Underflow!" << endl;
        return;
    }

    Node* temp = rear;

    cout << temp->data << " deleted from rear" << endl;

    rear = rear->prev;

    if (rear == nullptr) {
        front = nullptr;
    } else {
        rear->next = nullptr;
    }

    delete temp;
}
```

```
}
```

```
void getFront() {  
    if (isEmpty()) {  
        cout << "Deque is Empty!" << endl;  
    } else {  
        cout << "Front element: " << front->data << endl;  
    }  
}
```

```
void getRear() {  
    if (isEmpty()) {  
        cout << "Deque is Empty!" << endl;  
    } else {  
        cout << "Rear element: " << rear->data << endl;  
    }  
}
```

```
void display() {  
    if (isEmpty()) {  
        cout << "Deque is Empty!" << endl;  
        return;  
    }  
    Node* temp = front;  
    cout << "Deque elements: ";  
    while (temp != nullptr) {
```

```
        cout << temp->data << " ";  
        temp = temp->next;  
    }  
    cout << endl;  
}
```

```
~Deque() {  
    while (!isEmpty()) {  
        deleteFront();  
    }  
}  
};
```

```
int main() {  
    Deque dq;  
  
    dq.insertRear(10);  
    dq.insertRear(20);  
    dq.insertFront(5);  
    dq.insertFront(2);  
  
    dq.display();  
  
    dq.getFront();  
    dq.getRear();  
}
```

```

dq.deleteFront();

dq.display();


dq.deleteRear();

dq.display();


return 0;
}

```

```

Problems Output Debug Console Terminal Ports
10 inserted at rear
20 inserted at rear
5 inserted at front
2 inserted at front
Deque elements: 2 5 10 20
Front element: 2
Rear element: 20
2 deleted from front
Deque elements: 5 10 20
20 deleted from rear
Deque elements: 5 10
5 deleted from front
10 deleted from rear
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 3(CT-25169)>

```

Question no#03:

```

#include <iostream>

using namespace std;

struct Node {

    int data;

    Node* prev;

    Node* next;

    Node(int val) {

```

```
    data = val;
    prev = nullptr;
    next = nullptr;
}
};
```

```
class DoublyLinkedList {
public:
    Node* head;
    Node* tail;
```

```
    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }
```

```
    void insertEnd(int val) {
        Node* newNode = new Node(val);
        if (head == nullptr) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
}
```

```
void display() {  
    Node* temp = head;  
    while (temp != nullptr) {  
        cout << temp->data << " ";  
        temp = temp->next;  
    }  
    cout << endl;  
}  
};
```

```
DoublyLinkedList concatenate(DoublyLinkedList& L, DoublyLinkedList& M) {
```

```
    DoublyLinkedList N;
```

```
    Node* temp = L.head;
```

```
    while (temp != nullptr) {
```

```
        N.insertEnd(temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    temp = M.head;
```

```
    while (temp != nullptr) {
```

```
        N.insertEnd(temp->data);
```

```
        temp = temp->next;
```

```
}
```

```
return N;
```

```
}
```

```
int main() {
```

```
    DoublyLinkedList L;
```

```
    DoublyLinkedList M;
```

```
    for (int i = 2; i <= 10; i += 2) {
```

```
        L.insertEnd(i);
```

```
    }
```

```
    for (int i = 1; i <= 9; i += 2) {
```

```
        M.insertEnd(i);
```

```
    }
```

```
    cout << "List L (Evens): ";
```

```
    L.display();
```

```
    cout << "List M (Odds): ";
```

```
    M.display();
```

```

DoublyLinkedList N = concatenate(L, M);

cout << "List N (Concatenated L + M): ";

N.display();

return 0;
}

```

```

Problems Output Debug Console Terminal Ports
List L (Evens): 2 4 6 8 10
List M (Odds): 1 3 5 7 9
List N (Concatenated L + M): 2 4 6 8 10 1 3 5 7 9
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 3(CT-25169)>

```

Question no#04:

```

#include <iostream>

using namespace std;

struct Node {

    int data;

    Node* prev;

    Node* next;

    Node(int val) {

        data = val;

        prev = nullptr;

        next = nullptr;

    }

};

```

```
class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    void insertEnd(int val) {
        Node* newNode = new Node(val);
        if (head == nullptr) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }

    void display() {
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " ";
            temp = temp->next;
        }
    }
};
```

```
}  
  
cout << endl;  
}
```

```
void sortDescending() {  
    if (head == nullptr) return;  
  
    bool swapped;  
    Node* ptr1;  
    Node* lptr = nullptr;  
  
    do {  
        swapped = false;  
        ptr1 = head;  
  
        while (ptr1->next != lptr) {  
  
            if (ptr1->data < ptr1->next->data) {  
                int temp = ptr1->data;  
                ptr1->data = ptr1->next->data;  
                ptr1->next->data = temp;  
                swapped = true;  
            }  
            ptr1 = ptr1->next;  
        }  
        lptr = ptr1;  
    }  
}
```

```
        } while (swapped);  
    }  
};
```

```
DoublyLinkedList concatenate(DoublyLinkedList& L, DoublyLinkedList& M) {
```

```
    DoublyLinkedList N;
```

```
    Node* temp = L.head;
```

```
    while (temp != nullptr) {
```

```
        N.insertEnd(temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    temp = M.head;
```

```
    while (temp != nullptr) {
```

```
        N.insertEnd(temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    return N;
```

```
}
```

```
int main() {
```

```
    DoublyLinkedList L, M;
```

```

for (int i = 2; i <= 10; i += 2) L.insertEnd(i);

for (int i = 1; i <= 9; i += 2) M.insertEnd(i);

DoublyLinkedList N = concatenate(L, M);

cout << "List N before sorting: ";

N.display();

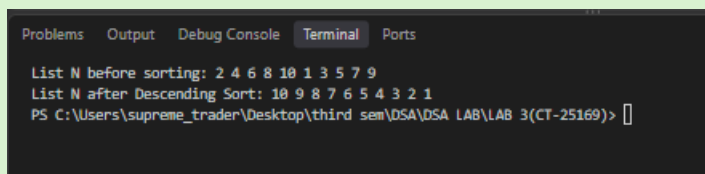
N.sortDescending();

cout << "List N after Descending Sort: ";

N.display();

return 0;
}

```



```

Problems Output Debug Console Terminal Ports
List N before sorting: 2 4 6 8 10 1 3 5 7 9
List N after Descending Sort: 10 9 8 7 6 5 4 3 2 1
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 3(CT-25169)>

```

Question no#05:

```

#include <iostream>

#include <string>

```

```
using namespace std;
```

```
struct Node {
```

```
    string url;
```

```
    Node* prev;
```

```
    Node* next;
```

```
    Node(string u) {
```

```
        url = u;
```

```
        prev = nullptr;
```

```
        next = nullptr;
```

```
    }
```

```
};
```

```
class BrowserHistory {
```

```
private:
```

```
    Node* curr;
```

```
public:
```

```
    BrowserHistory(string homepage) {
```

```
        curr = new Node(homepage);
```

```
    }
```

```
    void visit(string url) {
```

```
Node* newNode = new Node(url);  
newNode->prev = curr;
```

```
Node* temp = curr->next;  
while (temp != nullptr) {  
    Node* nextNode = temp->next;  
    delete temp;  
    temp = nextNode;  
}
```

```
curr->next = newNode;  
curr = newNode;  
}
```

```
string back(int steps) {  
    while (steps > 0 && curr->prev != nullptr) {  
        curr = curr->prev;  
        steps--;  
    }  
    return curr->url;  
}
```

```
string forward(int steps) {
```

```
while (steps > 0 && curr->next != nullptr) {  
    curr = curr->next;  
    steps--;  
}  
return curr->url;  
}  
};  
  
int main() {  
    BrowserHistory* browserHistory = new BrowserHistory("leetcode.com");  
  
    browserHistory->visit("google.com");  
    browserHistory->visit("facebook.com");  
    browserHistory->visit("youtube.com");  
  
    cout << "back(1): " << browserHistory->back(1) << endl;  
    cout << "back(1): " << browserHistory->back(1) << endl;  
    cout << "forward(1): " << browserHistory->forward(1) << endl;  
  
    browserHistory->visit("linkedin.com");  
  
    cout << "forward(2): " << browserHistory->forward(2) << endl;  
    cout << "back(2): " << browserHistory->back(2) << endl;  
    cout << "back(7): " << browserHistory->back(7) << endl;  
  
    return 0;  
}
```

}

```
Problems Output Debug Console Terminal Ports
back(1): facebook.com
back(1): google.com
forward(1): facebook.com
forward(2): linkedin.com
back(2): google.com
back(7): leetcode.com
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 3(CT-25169)>
```